4.3 Dynamic programming

Dynamic programming works on the principle of finding an overall solution by operating on an intermediate point that lies between where you are now and where you want to go.[2] The procedure is recursive, in that each next intermediate point is a function of the points already visited. A prototypical problem that is suitable for dynamic programming has the following properties:

- The problem can be decomposed into a sequence of decisions made at various stages.
- Each stage has a number of possible states.
- A decision takes you from a state at one stage to some state at the next stage.
- The best sequence of decisions (also known as a *policy*) at any stage is independent of the decisions made at prior stages.
- There is a well-defined cost for traversing from state to state across stages. Moreover, there is a recursive relationship for choosing the best decisions to make.

The method can be applied by starting at the goal and working backward to the current state. That is, we can first determine the best decision to make at the last stage. From there, we determine the best decision at the next to last stage, presuming we will make the best decision at the last stage, and so forth.

The simplest example is the *stagecoach problem* developed by Harvey Wagner [217]. The problem has a lot of window dressing involving a salesman taking his stagecoach through territory with hostile Native American Indians, but we'll dispense with that to cut straight to the core of the issue.

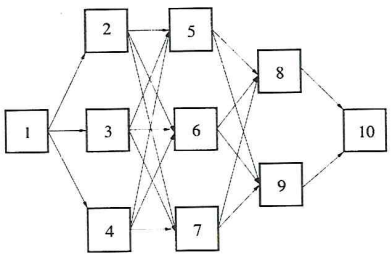


Fig. 4.6. The flow diagram indicating the choices for the salesman as he takes his stagecoach through hostile territory.

[2]The term *dynamic* was used to identify the approach as being useful for problems "in which times plays a significant role, and in which the order of operations may be crucial" [40].

The salesman must start from his current position and arrive at a fixed destination. He effectively has three stages at which he can make alternative choices about how to proceed (see figure 4.6). At the first stage, he has three alternative paths. Likewise, at the second stage, there are three alternatives. Finally, at the third stage there are two alternatives. The fourth stage does not offer a choice. The costs for traversing from each state at each stage to each next possible state are shown in figure 4.7. The problem is to find the least-cost path from the first state (1) to the last state (10).

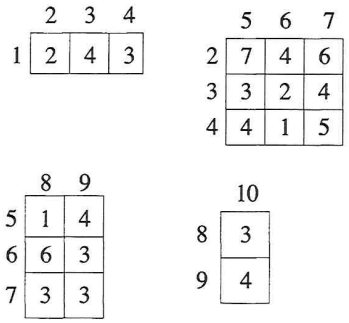


Fig. 4.7. The cost for making each possible choice at each decision stage. The row indicates the current state and the column indicates the next state. The four matrices correspond to the four stages. Note that only the first three stages offer a choice of which state to choose.

We need to introduce some notation here before proceeding to work out the answer:

- There are n stages and the decision of which state to choose at the n-th stage is x_n. Here, $n = 4$.
- $f_n(s, x_n)$ is the cost of the best sequence of decisions (policy) for all of the remaining stages, given that the salesman is in state s, stage n, and chooses x_n as the immediate next state.
- $x_n^*(s)$ is the value of x_n that minimizes $f_n(s, x_n)$ and $f_n^*(s)$ is the corresponding minimum value.
- The goal is to find $f_1^*(1)$ because the salesman is starting in state 1. This is accomplished by finding $f_4^*(s)$, $f_3^*(s)$, $f_2^*(s)$, and then finally $f_1^*(1)$.

What then is $f_4^*(s)$? For which x_4 is $f_4(s, x_4)$ minimized? This is almost a trick question because there's only one state to choose while in the fourth stage: $x_4 = 10$. Thus we easily compute the values shown below, and we have completed the first phase of the dynamic programming procedure.

s	$f_4^*(s)$	$x_4^*(s)$
8	3	10
9	4	10

The above matrix displays the cost of going from states 8 or 9 to a state at the final stage (in this case, there is only one such state).

The next phase asks for the value of $f_3^*(s)$. Recall that since the decisions are independent, $f_3(s, x_3) = c_{sx_3} + f_4^*(x_3)$, where c_{sx_3} is the cost of traveling from s to x_3. The matrix shown below indicates the relevant values of $f_3(s, x_3)$ for every possible choice of destination presuming s is 5, 6, or 7.

s	$f_3(s,8)$	$f_3(s,9)$	$f_3^*(s)$	$x_3^*(s)$
5	4	8	4	8
6	9	7	7	9
7	6	7	6	8

We can see, for example, that if we're in state 5, then state 8 minimizes the total remaining cost. State 9 minimizes the cost if $s = 6$, and state 8 minimizes the cost if $s = 7$.

Working backward again, we must find the value of $f_2^*(s)$. In an analogous manner to what appears directly above, the matrix shown below provides the relevant values of $f_2(s, x_2)$. Note that $f_2(s, x_2) = c_{sx_2} + f_3^*(x_2)$.

s	$f_2(s,5)$	$f_2(s,6)$	$f_2(s,7)$	$f_2^*(s)$	$x_2^*(s)$
2	11	11	12	11	5 or 6
3	7	9	10	7	5
4	8	8	11	8	5 or 6

Finally, with regard to $f_1^*(s)$, the last matrix offers the costs of proceeding to states 2, 3, or 4:

s	$f_1(s,2)$	$f_1(s,3)$	$f_1(s,4)$	$f_1^*(s)$	$x_1^*(s)$
1	13	11	11	11	3 or 4

This is the final table for the stagecoach problem. Now the dynamic programming method has been applied all the way back to the first stage. The values in the above matrix indicate the cost of traveling from state 1 to 2, 3, or 4.

At this point we can identify the best answer to the overall problem. In fact, here there are three best answers (policies), each with an equal cost of 11 units:

- $1 \rightarrow 3 \rightarrow 5 \rightarrow 8 \rightarrow 10$
- $1 \rightarrow 4 \rightarrow 5 \rightarrow 8 \rightarrow 10$
- $1 \rightarrow 4 \rightarrow 6 \rightarrow 9 \rightarrow 10$.

Note that the obverse procedure of choosing the least-cost path moving forward through the stages does not give the optimum solution. That greedy solution has a cost of 13 units (you can verify this). Thus, here is a case where the greedy approach fails, but dynamic programming generates the right answer. One drawback of the approach, however, is that it can be computationally intensive. If N is both the number of stages and states per stage, then the required number of operations scales as N^3. The method can, however, be extended to handle a variety of optimization problems including the order of multiplying

matrices and finding the best organization for a tree to minimize the cost of searching [425].

Dynamic programming algorithms tend to be somewhat complicated to understand. This is because, in practice, the construction of a dynamic program depends on the problem. It is a sort of "artistic intellectual activity depending in part on the specific structure of the sequential decision problem" [432]. This is why we'll illustrate this technique by two additional examples: matrix multiplication and the TSP.

Suppose that the dimensions of the matrices A_1, A_2, A_3, and A_4 are 20×2, 2×15, 15×40, and 40×4, respectively, and that we want to know the optimum way to compute $A_1 \times A_2 \times A_3 \times A_4$, i.e., we would like to compute this product with a minimum number of multiplications. Here we assume that to multiply two matrices, P, which is $n \times k$, by Q, which is $k \times m$, it takes nkm multiplications. The resulting matrix R is $n \times m$ and

$$r_{ij} = \sum_{v=1}^{k} p_{iv} q_{vj},$$

for all $1 \leq i \leq n$ and $1 \leq j \leq m$.
Note that different orders of multiplications have different costs. For example,

- $A(B(CD))$ requires $15 \cdot 40 \cdot 4 + 2 \cdot 15 \cdot 4 + 20 \cdot 2 \cdot 4 = 2680$ multiplications,
- $(AB)(CD)$ requires $20 \cdot 2 \cdot 15 + 15 \cdot 40 \cdot 4 + 20 \cdot 15 \cdot 4 = 4200$ multiplications, whereas
- $((AB)C)D$ requires $20 \cdot 2 \cdot 15 + 20 \cdot 15 \cdot 40 + 20 \cdot 40 \cdot 4 = 15800$ multiplications!

In a dynamic programming approach we create a structure $M(i, j)$ where we maintain a record of the minimum number of multiplications required to multiply matrices from A_i to A_j $(i \leq j)$. Clearly, $M(1,1) = M(2,2) = M(3,3) = M(4,4) = 0$, as no multiplications are required in these cases. Note that the problem involves finding $M(1,4)$.

The connection between solutions of the smaller problems and a bigger one is

$$M(i,j) = \min_{i \leq k < j} \{M(i,k) + M(k+1,j) + cost_{ij}^{k}\},$$

where $cost_{ij}^{k}$ is the number of multiplications required for multiplying the product $A_i \ldots A_k$ by $A_{k+1} \ldots A_j$. The point is that to multiply a sequence from A_i to A_j in the optimal way, we must find the optimal breaking point k such that the total number of multiplications required for calculating the product $A_i \ldots A_k$, which is $M(i,k)$, the product $A_{k+1} \ldots A_j$, which is $M(k+1,j)$, and these two products together (which is $cost_{ij}^{k}$) is minimal.

Keeping this in mind, we can proceed as follows. It's a basic exercise to get:

$$M(1,2) = 600,$$
$$M(2,3) = 1200,$$
$$M(3,4) = 2400,$$

because there's no room for a breaking point in multiplying two matrices. Then,

$$M(1,3) = 2800$$
$$M(2,4) = 1520.$$

Note that $M(1,3)$ is the smaller of two possibilities

$$M(1,1) + M(2,3) + cost_{13}^1 = 0 + 1200 + 1600 = 2800, \text{ and}$$
$$M(1,2) + M(3,3) + cost_{13}^2 = 600 + 0 + 12000 = 12600.$$

Similarly, $M(2,4)$ is the smaller of two possibilities

$$M(2,2) + M(3,4) + cost_{24}^2 = 0 + 2400 + 120 = 2520, \text{ and}$$
$$M(2,3) + M(4,4) + cost_{24}^3 = 1200 + 0 + 320 = 1520.$$

Finally, we find

$$M(1,4) = 1680,$$

as the smallest value from three possibilities

$$M(1,1) + M(2,4) + cost_{14}^1 = 0 + 1520 + 160 = 1680,$$
$$M(1,2) + M(3,4) + cost_{14}^2 = 600 + 2400 + 1200 = 4200, \text{ and}$$
$$M(1,3) + M(4,4) + cost_{14}^3 = 2800 + 0 + 3200 = 6000.$$

Thus, the minimal cost in terms of the number of multiplications is 1680, but we still need to find the corresponding order of matrix multiplications. To find this best order we need an additional data structure, $O(i,j)$, where we keep the index of the best breaking point. In other words, $O(i,j) = k$ if and only if $M(i,j)$ attains the minimum value for $M(i,k)+M(k+1,j)+cost_{ij}^k$. The indices kept in the matrix O reveal the order we were searching for: $A_1((A_2A_3)A_4)$.

Let's conclude this section by considering an example of a dynamic program for the TSP. It's a lengthy example, so be forewarned. To make things meaningful, let's tackle a five-city TSP. The matrix L of distances between cities is given below.

$$L = \begin{bmatrix} 0 & 7 & 12 & 8 & 11 \\ 3 & 0 & 10 & 7 & 13 \\ 4 & 8 & 0 & 9 & 12 \\ 6 & 6 & 9 & 0 & 10 \\ 7 & 7 & 11 & 10 & 0 \end{bmatrix}$$

The cost of traveling from city i to city j is given in the i-th row and j-th column of the matrix L. For example, the distance between cities 1 and 3 is $L(1,3) = 12$. This distance is different than the distance between cities 3 and 1, which is $L(3,1) = 4$; therefore, we're facing an asymmetric TSP. Note that $L(i,i) = 0$ for all $1 \leq i \leq n$. Figure 4.8 illustrates the case.

The tour for the TSP is a cycle that visits *every* city once and only once, so the starting city is not of any real significance. Suppose we start from city 1. Now, we're ready to split the problem into smaller problems. Let $g(i,S)$ denote the length of the shortest path from city i to city 1 that passes through *each* city listed in the set S exactly once. Thus, in particular,

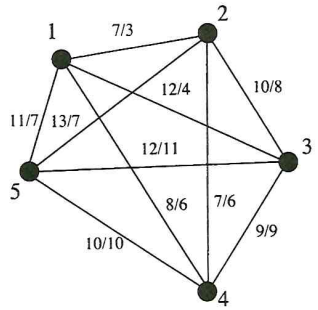


Fig. 4.8. A TSP with five cities. For each pair of cities, two numbers are given separated by a slash. The first number provides the distance from the city with the lower index to the city with the larger index, whereas the second number represents the distance in the opposite direction.

$$g(4, \{5, 2, 3\})$$

represents the shortest path that leads from city 4 through cities 5, 2, and 3 (in some unspecified order) and then returns to city 1. To find the length of the shortest complete tour for the TSP we need to find

$$g(1, V - \{1\}),$$

where V represents the set of *all* the cities for the TSP. In other words, we're trying to find the shortest path that starts from city 1, visits every city in the set $V - \{1\}$ exactly once, and returns to city 1.

The dynamic programming formulation provides a connection between solutions of the smaller problems and a bigger one. In general, we claim that

$$g(i, S) = \min_{j \in S} \{L(i, j) + g(j, S - \{j\})\}.$$

In other words, to select the shortest path that originates at city i and visits every city in S before returning to city 1, we have to find a city j from the set S such that the sum of $L(i, j)$ and its continuation, $g(j, S - \{j\})$, is minimized. If we know the solutions for the smaller problems (i.e., the problems where the cardinality of S is k), we can obtain a solution for a bigger problem (i.e., where the cardinality of S is $k + 1$).

Let's return to the example. The problem is to find

$$g(1, \{2, 3, 4, 5\}).$$

Note also that

$$g(2, \emptyset) = L(2, 1) = 3,$$
$$g(3, \emptyset) = L(3, 1) = 4,$$
$$g(4, \emptyset) = L(4, 1) = 6, \text{ and}$$
$$g(5, \emptyset) = L(5, 1) = 7,$$

since the set S is empty and we proceed directly from the city i ($i = 2, 3, 4, 5$) to city 1. The next iteration is also straightforward: we need to find the solutions to all the problems where the cardinality of S is one. This would involve solving 12 subproblems, since we'll start from cities 2, 3, 4, and 5, and for each of these cities, we'll consider three possible singleton sets. For example, for city 2 we have

$$\begin{aligned} g(2, \{3\}) &= L(2,3) + g(3, \emptyset) = 10 + 4 = 14, \\ g(2, \{4\}) &= L(2,4) + g(4, \emptyset) = 7 + 6 = 13, \text{ and} \\ g(2, \{5\}) &= L(2,5) + g(5, \emptyset) = 13 + 7 = 20. \end{aligned}$$

Note that the result $g(2, \{3\}) = 14$ means that the length of the shortest path from city 2 to 1, which before arriving at 1 goes through city 3, is 14. This result is obvious since S consists of only a single element and there is no room for any choices. Similarly, for city 3 we have

$$\begin{aligned} g(3, \{2\}) &= L(3,2) + g(2, \emptyset) = 8 + 3 = 11, \\ g(3, \{4\}) &= L(3,4) + g(4, \emptyset) = 9 + 6 = 15, \\ g(3, \{5\}) &= L(3,5) + g(5, \emptyset) = 12 + 7 = 19, \end{aligned}$$

for city 4,

$$\begin{aligned} g(4, \{2\}) &= L(4,2) + g(2, \emptyset) = 6 + 3 = 9, \\ g(4, \{3\}) &= L(4,3) + g(3, \emptyset) = 9 + 4 = 13, \\ g(4, \{5\}) &= L(4,5) + g(5, \emptyset) = 10 + 7 = 17, \end{aligned}$$

and for city 5,

$$\begin{aligned} g(5, \{2\}) &= L(5,2) + g(2, \emptyset) = 7 + 3 = 10, \\ g(5, \{3\}) &= L(5,3) + g(3, \emptyset) = 11 + 4 = 15, \\ g(5, \{4\}) &= L(5,4) + g(4, \emptyset) = 10 + 6 = 16. \end{aligned}$$

Now we're ready for the next iteration of the dynamic program where the cardinality of S is two. In this iteration we have to solve 12 subproblems as well, since each of four cities can be considered along with any two other cities (out of three). Thus, for city 2 we have

$$\begin{aligned} g(2, \{3,4\}) &= \min\{L(2,3) + g(3, \{4\}), L(2,4) + g(4, \{3\})\} \\ &= \min\{10 + 15, 7 + 13\} = \min\{25, 20\} = 20, \\ g(2, \{3,5\}) &= \min\{L(2,3) + g(3, \{5\}), L(2,5) + g(5, \{3\})\} \\ &= \min\{10 + 19, 13 + 15\} = \min\{29, 28\} = 28, \\ g(2, \{4,5\}) &= \min\{L(2,4) + g(4, \{5\}), L(2,5) + g(5, \{4\})\} \\ &= \min\{7 + 17, 13 + 16\} = \min\{24, 29\} = 24. \end{aligned}$$

The result $g(2, \{3,4\}) = 20$ means that the length of the shortest path from city 2 to 1, which before arriving at 1 goes through cities 3 and 4 (in some order), is 20. This time we have to consider two possibilities (going first to city 3 or going first to city 4), and select the better option. Similarly, for city 3,

$$
\begin{aligned}
g(3,\{2,5\}) &= \min\{L(3,2)+g(2,\{5\}), L(3,5)+g(5,\{2\})\}\\
&= \min\{8+20, 12+10\} = \min\{28,22\} = 22,\\
g(3,\{2,4\}) &= \min\{L(3,2)+g(2,\{4\}), L(3,4)+g(4,\{2\})\}\\
&= \min\{8+13, 9+9\} = \min\{21,18\} = 18,\\
g(3,\{4,5\}) &= \min\{L(3,4)+g(4,\{5\}), L(3,5)+g(5,\{4\})\}\\
&= \min\{9+17, 12+16\} = \min\{26,28\} = 26,
\end{aligned}
$$

for city 4,

$$
\begin{aligned}
g(4,\{2,3\}) &= \min\{L(4,2)+g(2,\{3\}), L(4,3)+g(3,\{4\})\}\\
&= \min\{6+14, 9+15\} = \min\{20,24\} = 20,\\
g(4,\{2,5\}) &= \min\{L(4,2)+g(2,\{5\}), L(4,5)+g(5,\{2\})\}\\
&= \min\{6+20, 10+10\} = \min\{26,20\} = 20,\\
g(4,\{3,5\}) &= \min\{L(4,3)+g(3,\{5\}), L(4,5)+g(5,\{3\})\}\\
&= \min\{9+19, 10+15\} = \min\{28,25\} = 25.
\end{aligned}
$$

and for city 5,

$$
\begin{aligned}
g(5,\{2,3\}) &= \min\{L(5,2)+g(2,\{3\}), L(5,3)+g(3,\{2\})\}\\
&= \min\{7+14, 11+11\} = \min\{21,22\} = 21,\\
g(5,\{2,4\}) &= \min\{L(5,2)+g(2,\{4\}), L(5,4)+g(4,\{2\})\}\\
&= \min\{7+13, 10+19\} = \min\{20,29\} = 20,\\
g(5,\{3,4\}) &= \min\{L(5,3)+g(3,\{4\}), L(5,4)+g(4,\{3\})\}\\
&= \min\{11+15, 10+13\} = \min\{26,23\} = 23.
\end{aligned}
$$

Now we're ready for the next step, where the cardinality of the set S is three. Here we face only four subproblems: one for each city. Thus, for city 2 we have to select the shortest path among three paths:

$$
\begin{aligned}
&g(2,\{3,4,5\})\\
&\quad = \min\{L(2,3)+g(3,\{4,5\}), L(2,4)+g(4,\{3,5\}), L(2,5)+g(5,\{3,4\})\}\\
&\quad = \min\{10+26, 7+25, 13+23\} = \min\{36,32,34\} = 32.
\end{aligned}
$$

The result $g(2,\{3,4,5\}) = 32$ means that the length of the shortest path from city 2 to 1, which before arriving at 1 goes through cities 3, 4, and 5 (in some order), is 32. Now we have to consider three possibilities, going first to city 3, 4, or 5, and select the best option. Similarly, for cities 3, 4, and 5 we have

$$
\begin{aligned}
&g(3,\{2,4,5\})\\
&\quad = \min\{L(3,2)+g(2,\{4,5\}), L(3,4)+g(4,\{2,5\}), L(3,5)+g(5,\{2,4\})\}\\
&\quad = \min\{8+24, 9+20, 12+20\} = \min\{32,29,32\} = 29,\\
&g(4,\{2,3,5\})\\
&\quad = \min\{L(4,2)+g(2,\{3,5\}), L(4,3)+g(3,\{2,5\}), L(4,5)+g(5,\{2,3\})\}\\
&\quad = \min\{6+28, 9+22, 10+21\} = \min\{34,31,31\} = 31, \text{ and}\\
&g(5,\{2,3,4\})\\
&\quad = \min\{L(5,2)+g(2,\{3,4\}), L(5,3)+g(3,\{2,4\}), L(5,4)+g(4,\{2,3\})\}\\
&\quad = \min\{7+20, 11+18, 10+20\} = \min\{27,29,30\} = 27.
\end{aligned}
$$

Now, it's time for the final iteration where we start from city 1 and close the cycle. Thus we return to the original problem:

$$g(1, \{2,3,4,5\}) = ?$$

Starting from city 1 we have four options: to go first either to city 2, city 3, city 4, or city 5. The other subproblems (i.e., the shortest lengths for each continuation of each possible beginning) are already known, so it's easy to make the final decision:

$$\begin{aligned} g(1, \{2,3,4,5\}) = \min\{&L(1,2)+g(2,\{3,4,5\}), L(1,3)+g(3,\{2,4,5\}), \\ &L(1,4)+g(4,\{2,3,5\}), L(1,5)+g(5,\{2,3,4\})\} = \\ &\min\{7+32, 12+29, 8+31, 11+27\} = \min\{39,41,39,38\} = 38. \end{aligned}$$

Thus the shortest tour has a length of 38.

Which tour is that? As with the matrix multiplication problem, we've found the optimum value of the evaluation function but we still have to find out which tour has given us this result. Again, this is easy if we keep track of our calculations. We need an additional data structure, W, that provides information on the next city that should be chosen to minimize a path. Thus, for example,

$$W(5, \{2,3,4\}) = 2,$$

since the shortest path from city 5 to city 1, which has to pass through cities 2, 3, and 4 before arriving at 1, must go to city 2 first. Similarly,

$$W(1, \{2,3,4,5\}) = 5.$$

Thus, the global solution for this TSP is given by the tour

$$1 - 5 - 2 - 4 - 3 - 1,$$

and the length of this tour is, of course,

$$11 + 7 + 7 + 9 + 4 = 38.$$

Does the thought of using dynamic programming to solve a 50-city TSP strike fear in your heart?

4.4 Branch and bound

We've seen that the size of real-world problems can grow very large as the number of variables in the problem increases. Recall that there are $(n-1)!/2$ different solutions for the symmetric TSP. Exhaustive search is out of the question for $n > 20$, so it would be helpful to have some heuristic that eliminates parts of the search space where we know that we won't find an optimum solution.

Branch and bound is one such heuristic that works on the idea of successive partitioning of the search space. We first need some means for obtaining a lower bound on the cost for any particular solution (or an upper bound depending on